"An Introduction to Prolog"
Part II

Viviana Mascardi
viviana.mascardi@unige.it

University of Genoa – DIBRIS

Dibris

Disclaimer

- This presentation may contain material protected by copyright laws.
- In particular, these lessons are based on the material that Peter Schachte made available on the Web years ago, as part of the Logic and Computation Course at the Computer Science Department of the University Melbourne
- We made this material available on the Web only to ensure timely dissemination among our students, and is meant only for students' personal use.
- Any use different from the students' personal use is prohibited.
- V. Mascardi thanks Professor Schachte for the courtesy of sharing his teaching material on Prolog.

Dibris

Outline

1. Lists
   - List Membership
   - Member in Action
   - List Concatentation
   - Append in Action

Dibris

Outline

1. Lists
   - List Membership
   - Member in Action
   - List Concatentation
   - Append in Action

Dibris

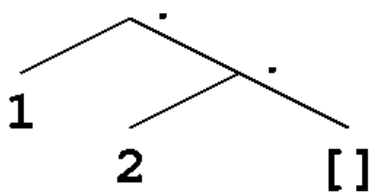
Lists

Lists are very widely used in Prolog — they have a special syntax

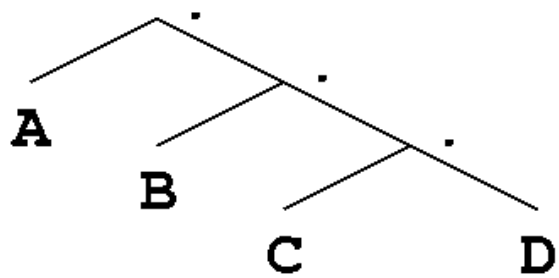
```
list([]).
list([_|Tail]) :- list(Tail).
```

- `[]` is the empty list.
- `[H|T]` is the list with head `H` and tail `T`.

`[1,2]`



`[A,B,C|D]`



List functor is `.` (dot), arity is 2
Bracket notation is special syntax, but lists are ordinary terms

Dibris

List Membership

The `member`/2 predicate can check list membership
It is a built-in of SWI Prolog, but could be defined as:

```
member(X, [X|_]).      % X is member of list beginning w
member(X, [_|L]) :-    % X is member of list
    member(X, L).      % if it is member of tail
```

Member in Action

Member can be used in different ways:

```
?- member(c, [a,b,c,d]).
Yes
?- member(e, [a,b,c,d]).
No
?- member(X, [a,b,c]).
X = a ;
X = b ;
X = c ;
No
?- L = [_,_,_], member(a, L).
L = [a, _G291, _G294] ;
L = [_G288, a, _G294] ;
L = [_G288, _G291, a] ;
No
```

List Concatentation

The `append`/3 predicate concatenates (`++` in Haskell) two lists
Again a built-in, but could be defined as:

```
append([], L, L).                %[] ++ L gives L
append([J|K], L, [J|KL]) :- %to ++ a list with first
                            %element J and remainder
                            %K to list L
  append(K, L, KL).              %determine K ++ L and
                            %then add J to front
```

Recall Haskell definition:

```
append [] l = l
append (j:k) l = j : append k l
```

The same, but can only append lists

Append in Action

Prolog's `append` is more flexible:

```
?- append([a,b,c], [d,e], L).
L = [a, b, c, d, e] ;
No

?- append(L, [d,e], [a,b,c,d,e]).
L = [a, b, c] ;
No

?- append([a,b,c], L, [a,b,c,d,e]).
L = [d, e] ;
No
```

```
?- append(K, L, [a,b]).
K = []
L = [a, b] ;

K = [a]
L = [b] ;

K = [a, b]
L = [] ;

No
```

Dibris